

AI サービス連携による 技術学習・開発効率化事例

部内向けフィードバック資料

Abstract

Python 学習、AI・データ分析関連の技術学習、資格試験対策および技術調査を目的として、複数の AI サービスを約 2 か月利用した。

ChatGPT、Perplexity、Cursor を中核に、Codex、NotebookLM、Google Colaboratory、GitHub、Obsidian を組み合わせることで、調査から学習、開発、レビュー、検証、知識蓄積までの一連の流れを構築した。

実際に利用した結果、AI サービスは単体で利用するよりも、得意分野ごとに役割を分担して連携利用する方が効果的だと感じた。特に、ChatGPT による方針整理、Perplexity による専門調査、Cursor による実装、Codex によるレビュー、GitHub による成果物共有を組み合わせることで、各サービスの得意分野を生かしながら作業を進めることができた。

主な役割分担

- **Perplexity** : 技術調査、論文・専門情報の収集、画像入力による数式・考え方の確認
- **ChatGPT** : 全体統括、学習計画、技術相談、開発方針整理、サンプルコード・Notebook 作成、他サービスへの依頼文作成
- **Cursor (PC)** : 実装、デバッグ、リファクタリング、モジュール化、ターミナル操作、パッケージ管理、GitHub への反映
- **Cursor iOS (予定)** : モバイル端末から Agent へ指示し、コード修正・確認・進捗確認を試す予定
- **Codex** : PC・モバイルの双方から利用し、GitHub 上のコード確認、修正、コードレビュー
- **Google Colaboratory** : モバイル環境での Notebook 実行、軽い修正・検証
- **GitHub** : コード、Notebook、Markdown、実験成果物などを各開発環境間で共有するハブ
- **NotebookLM** : 与えた資料の整理・要約、複数資料の統合
- **Obsidian** : Markdown ノートの編集、追記、知識の整理・体系化

1. AI サービス連携図

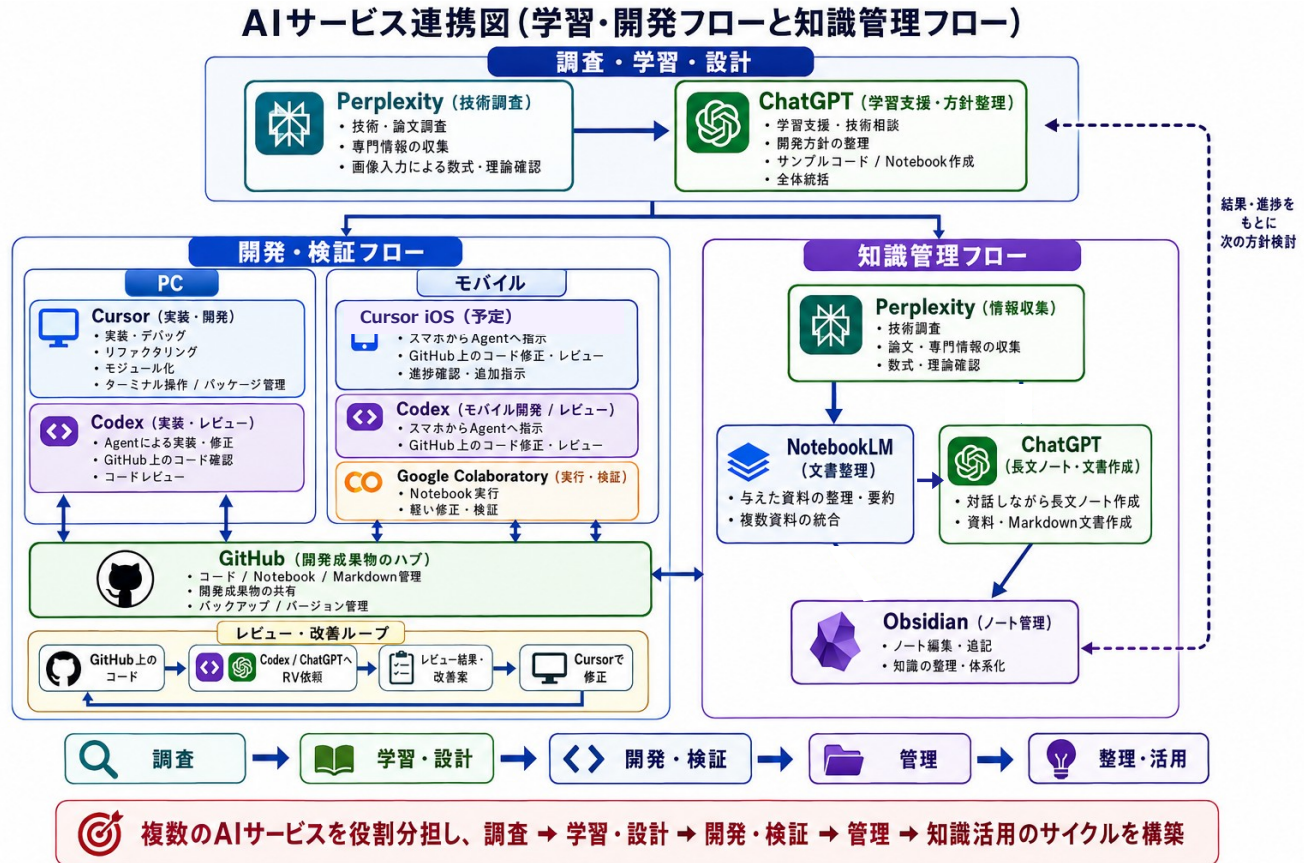


Figure 1. AI サービス連携図 (学習・開発フローと知識管理フロー)

図では、開発・検証と知識管理を並列の流れとして示す。PC / モバイルは作業環境の区分であり、レビュー依頼は端末間ではなく Codex や ChatGPT に対して行う。

2. 利用サービス一覧

今回利用したサービスと主な役割、継続利用評価を以下に示す。

サービス	主な役割	評価
ChatGPT	全体統括、学習支援、技術相談、開発方針整理、文書・Notebook 作成、他 AI への依頼文作成	◎ 継続推奨
Perplexity	技術・論文調査、専門情報収集、画像入力による数式・考え方の確認	○ 条件付き推奨
Cursor	PC での実装、デバッグ、コード整理、開発環境操作	◎ 継続推奨
Cursor iOS（予定）	スマートフォンから Agent へ指示する予定 コード修正・確認・進捗確認を想定	○ 利用予定
Codex	GitHub 上のコード確認・修正、コードレビュー	○ 継続利用
NotebookLM	文書整理、要約、複数資料の統合	△ 無料版で十分
Google Colaboratory	Notebook 実行、軽いコード修正・検証	△ 無料版で十分
GitHub	コード、Notebook、Markdown、実験成果物の共有・管理	○ 無料版で十分
Obsidian	Markdown ベースのノート管理、追記、知識整理	○ 無料版で十分

GitHub 成果物は口頭説明で補足する。

参照先：<https://github.com/astrare2480?tab=repositories>

3. 各サービスの特徴

3.0 サービスの位置づけ

各サービスは同じ種類の製品ではないため、単純な性能比較ではなく「一言でいうと何か」「今回どこで使ったか」という役割で整理する。

サービス	一言でいうと	今回の主な役割	他サービスとの違い
ChatGPT	何でも相談できる全体統括 AI	学習計画、方針整理、文書作成、他 AI への依頼文作成	個別作業よりも全体の整理・相談役として利用
Perplexity	技術調査に強い検索 AI	論文・専門情報の調査、数式や考え方の確認	情報源を探し、複数の情報を比較する用途に強い
Cursor	AI エージェント付き開発環境	PC で実装、修正、デバッグ、開発環境操作	回答だけでなく実際のコードや開発環境を操作
Cursor iOS（予定）	スマホから Agent へ指示予定	移動中の補助利用を想定	PC 作業をモバイルから補助する想定
Codex	コード確認・レビュー担当 AI	GitHub 上のコードレビュー、修正後の再確認	Cursor とは別の視点で第三者レビューを実施
NotebookLM	与えた資料の整理が得意な AI	複数資料の要約・統合	外部調査より、渡した資料の整理に向く
Google Colaboratory	Notebook をブラウザで実行する環境	モバイルで Notebook を実行・確認	AI そのものではなく、実行・検証のための環境
GitHub	コード・成果物の構成管理サービス	PC・モバイル・AI サービスをつなぐ共通ハブ	複数サービスが同じ成果物を見るための保存先
Obsidian	Markdown ノート管理ツール	AI が作ったノートの追記・知識化	最終的な知識整理を人間側で行う場所

使い分けの中心は「ChatGPT＝全体統括」「Perplexity＝調査」「Cursor＝実装」「Codex＝レビュー」「GitHub＝共通ハブ」である。NotebookLM・Colab・Obsidian は用途に応じて補助的に利用した。

3.0.1 利用プラン・料金

※料金は 2026 年 8 月の利用時点で確認した内容を参考に記載している。料金・利用上限は変更される可能性がある。

サービス	今回の利用プラン	利用時点の料金	利用上限・所感
ChatGPT	Plus	3,000 円/月	通常利用は定額。Codex など一部機能には利用上限がある
Perplexity	Pro	20 米ドル/月	通常の質問は定額。高度な調査機能では利用量を意識する
Cursor	Pro	20 米ドル/月	利用上限付きだが、今回の個人開発では余裕があった
Cursor iOS（予定）	未利用（今後利用予定）	追加料金なし（予定）	PC 版 Cursor の補助として利用予定
Codex	ChatGPT Plus に含む	追加料金なし	週次利用枠があり、重いレビューでは消費を意識する
NotebookLM	無料	0 円	現状の用途では無料版で十分
Google Colaboratory	無料	0 円	現状の用途では無料版で十分
GitHub	Free	0 円	個人用途では無料版で十分
Obsidian	無料利用	0 円	現状の用途では無料版で十分

3.1 ChatGPT

生成 AI サービスとして、学習計画、模擬試験問題作成、技術相談、サンプルコード生成、開発方針の整理、各 AI サービスの利用方針検討、各サービスへの依頼文作成などに利用した。

特に便利だったのは、別のチャットで扱った内容を踏まえながら継続的に相談できることである。学習内容や進捗を前提として次に何を行うか相談できるため、今回利用した AI サービス群の中では、全体を統括する役割として利用した。

資格試験学習では、学習スケジュールの作成や模擬問題の生成に利用した。開発では、サンプルコードや Jupyter Notebook 形式の教材を作成し、それを Cursor で実装・修正し、Codex や ChatGPT でレビューする流れを作ることができた。

また、長い学習ノートや技術ノートの生成・編集にも利用した。以前は Canvas を利用し、通常のチャット回答として断片的に文章を生成するよりも、長い文書を一つの成果物としてまとめる用途で特に便利だった。

最初から完成形を細かく指定するだけでなく、「項目を追加する」「構成を変える」「図を直す」「文章を修正する」といった変更を会話しながら積み重ねることができる。本資料についても、構成の検討、連携図の作成、文章の修正などを ChatGPT との対話を通して行っている。

さらに、開発結果や GitHub 上の成果物、Obsidian へ蓄積した内容をもとに、次に何を実装・調査するかを再度 ChatGPT へ相談することで、開発・学習サイクル全体の方針検討にも利用した。

利用したプランは ChatGPT Plus である。定額制で通常の ChatGPT 機能を利用でき、今回利用した Codex も同じ契約内で使用している。一方で、高負荷な機能には利用上限があるため、完全な無制限利用ではない。

また、カスタム指示やメモリを設定することで、回答の方向性を個人向けに調整できる。

3.1 ChatGPT — 相談しながら方針・文章を整理する

学習計画、技術相談、教材・ノート作成、他のAIに出す依頼文の整理に利用した。

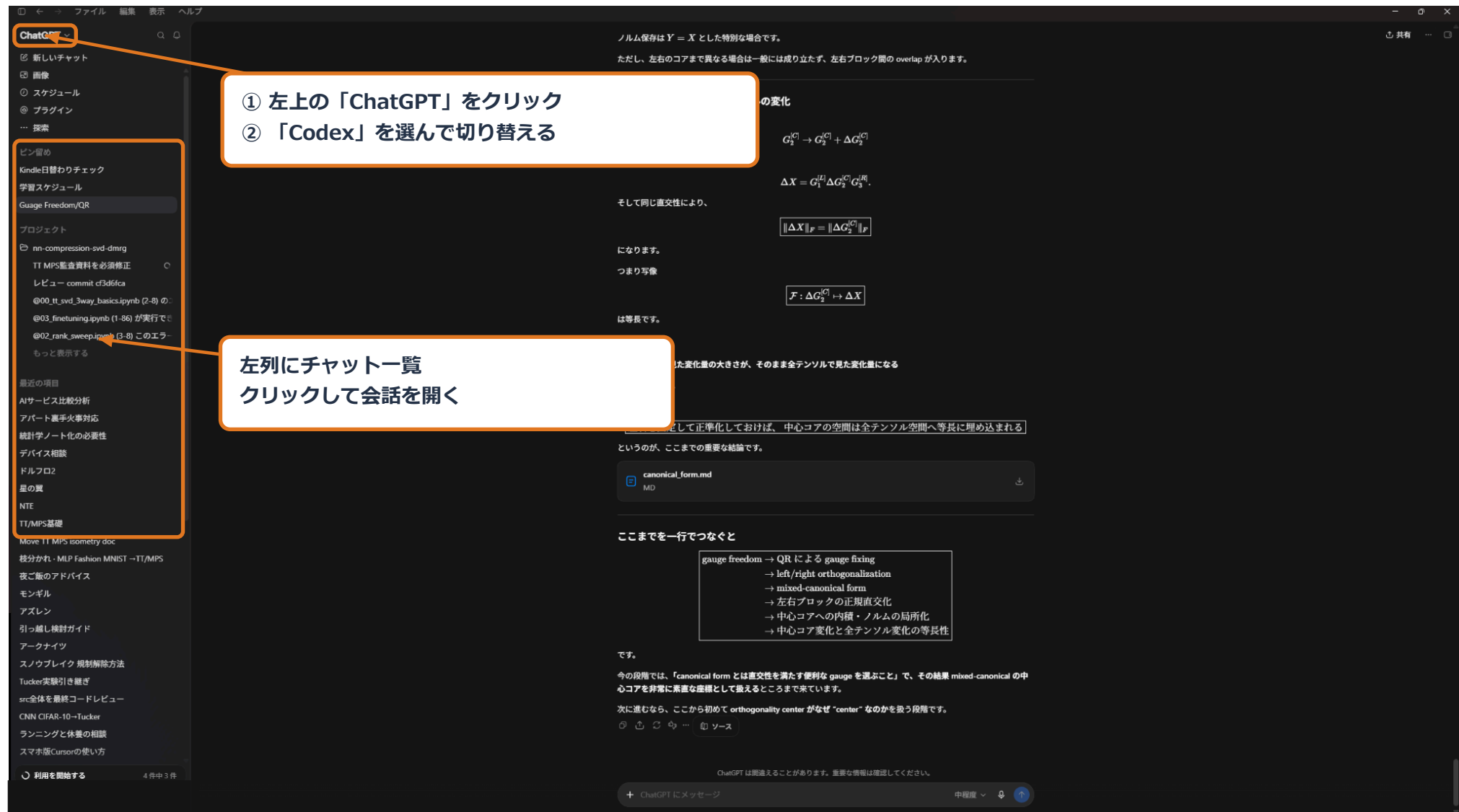


図2 左上でChatGPT / Codexを切り替える。操作の公式説明

3.1 ChatGPT — パーソナライズとレビュー依頼

パーソナライズ

Codex の指示

すべてのチャットで Codex に追加の指示やコンテキストを設定します。リポジトリの指示も適用される場合があります。 [詳細を見る](#)

保存する

大学は物理を専攻し、専門は物性物理学だった。
研究としては、DMRGを用いて、スピン鎖の基底状態のエネルギーを求めている。
所持資格がネットワークスペシャリスト、伝送交換主任技術者、線路主任技術者、CCNP、AI実装検定A級、pythonデータ分析認定試、第一級陸上無線技術士なので、技術系の質問をする。
データ・サイエンスに関しては
データ分析には必ず Python (pandas/scikit-learn) を使用してください。
数値計算手法の紹介の際には必ず例を載せてください。
特に指定がない限り、可視化は matplotlib か seaborn を使ってください。
ソースコードにはコメントを付与してください。
コードは、コピペしてそのままcursorで動く形式で出力してください。
コードのマークダウンには、display数式を $...$ に変更し、添字・演算子・積の間隔も整理して。
質問の回答の数学及びソースコード背景についても、必要であれば補足してください。

学習用Notebookを作るときは、完成済みの教材にしすぎず、ユーザー本人が手を動かす余地を必ず残す。

特に以下を守る。

- 学習用Notebookでは、実装部分をすべて完成させず、適切な箇所に「TODO」を残す。
 - コードの答えを先に全部書かず、「このセルで何を確認するのか」「数式上は何を実装するのか」「期待する検証結果」までを示し、実装自体はユーザーが行える構成にする。
 - 1つのNotebookで先の内容まで進みすぎない。その時点で学んでいるテーマの範囲で止める。
 - 添付資料や既存ロードマップに「ここで止める」「次は別Notebook」と書かれている場合、それを優先し、勝手に次の内容まで実装しない。
 - TT/MPS基礎では、理論 → 小さい数値例 → TODO実装 → 数値検証、の順で進める。
 - Jupyter Notebook / VS Code のMarkdown数式は、インラインを $...$ 、別行数式を $...$ で記述する。 $...$ や $...$ は使わない。
 - Notebook作成後は、数式デリミタが崩れていないことと、Run All が通ることを確認する。
 - ユーザーは「完成コードを読む」より「自分で実装して理解する」学習スタイルを優先している。
- 2026年9月、Gauge freedom / QR分解のNotebookで、回答側が第2コアのQRや $...$ まで完成実装してしまい、ユーザーが手を動かす部分がほぼ無くなった。また、Markdown数式の形式も一度誤った。この2点を今後繰り返さない。

自分に合った回答を依頼する

専門分野、回答の詳しさ、コードへのコメント、学習用教材の作り方などを指示する。

学習・開発の結果を戻し、次に何をするか継続して相談した。長い文書も「項目を足す」「構成を直す」と会話を重ねて整理できた。

ChatGPTが作成したレビュー依頼の形式

1. 目的：次の作業へ進めてよいか判断してもらう。
2. 対象：確認するコード・Notebook・結果を指定。
3. 制約：レビューだけ行い、勝手に変更しない。
4. 観点：コードの正しさ、比較方法、再現性、成果物管理。
5. 分類：重大・重要・軽微・文書上の問題に分ける。
6. 判定：A=次へ進める、B=修正後に進める、C=修正・再確認が必要。

添付画面の見出しは「Codexの指示」。
共通アプリの設定例として掲載する。

3.2 Perplexity

Perplexity は、検索結果や出典を確認しながら技術情報を調べる用途で利用した。IT 分野に限らず、論文や専門的な情報を複数の情報源からまとめて確認したい場合に便利だった。

Python ライブラリの使用方法、AI・機械学習に関する技術的な疑問、ニューラルネットワークの軽量化や計算量削減に関する専門技術などの調査に活用した。

特に、通常の検索エンジンで複数ページを個別に確認する代わりに、関連する技術情報や論文をまとめて探し、その内容と出典を確認する用途で便利だった。

また、紙のノートに書いた数式や考え方をスマートフォンで撮影し、写真を添付して、自分の式変形や理論上の考え方が妥当か確認する用途にも利用した。単なる情報検索だけでなく、自分で考えた内容を検証する補助としても利用できる点が便利だった。

専門的な調査では、過去に自分が扱った分野と重なる内容もあり、AI の回答をそのまま採用するのではなく、自分で妥当性を確認しながら利用した。

一方、資格試験の一般的な内容や、比較的広く知られているプログラミング情報を調べるだけであれば、無料版や他の AI サービスでも十分な場面が多いと感じた。

Perplexity Pro では、通常の実索に加えて Pro Search や Deep Research など、より多くの情報源を使って調査する機能を利用できる。日常的な狭い技術質問には Pro Search、複数の研究や手法を横断して比較し、今後の方針を決めるような調査には Deep Research が向いていると感じた。

また、ChatGPT と同様にパーソナライズ設定が可能であり、回答の前提や自分の技術レベルを指定できる。

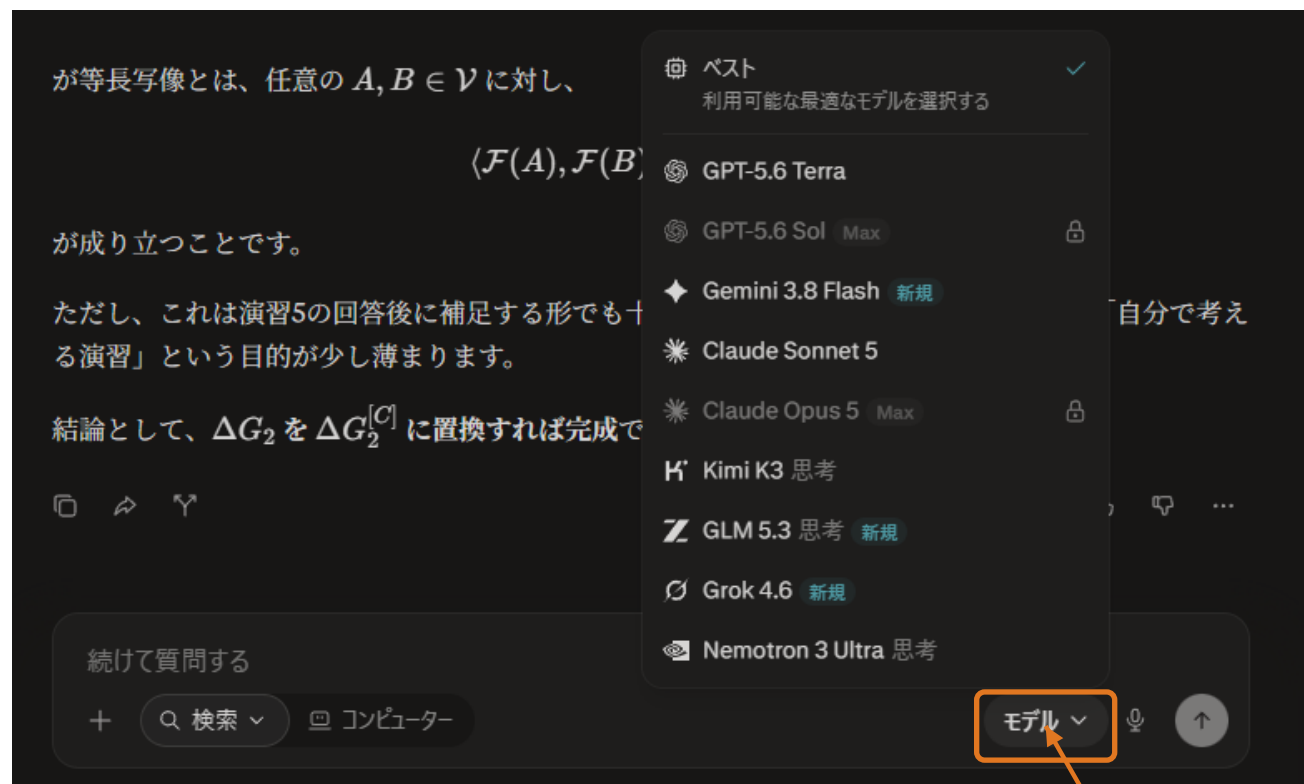
3.2 Perplexity — 出典を確認しながら調べる

技術情報・論文の検索や、写真で渡した数式・考え方の確認に利用した。



図3 Computerは調査結果の整理・成果物作成にも対応。Computerの公式説明

3.2 Perplexity — モデルはプランに応じて選択



モデル選択の読み方

「ベスト」は利用可能なモデルから自動選択。

この画面ではGPT-5.6 Terra、Gemini 3.8 Flash、Claude Sonnet 5、Kimi K3などが並ぶ。

Proプランでは鍵マーク付きのモデルは選択できない。利用するにはMaxなどの上位プランが必要。

ここを押してモデル一覧を開く

通常の検索ではモデルを選べる。Researchは調査用の組み合わせを自動選択するため、個別モデルを指定できない。

参考：Maxの公式説明 / Researchの公式説明（2026年9月14日確認）

3.2 Perplexity — パーソナライズで回答方針を指定

カスタム指示

大学は物理を専攻し、専門は物性物理学だった。
研究としては、DMRGを用いて、スピン鎖の基底状態のエネルギーを求めている。
所持資格がネットワークスペシャリスト、伝送交換主任技術者、線路主任技術者、CCNP、AI実装検定A級、python
データ分析認定試、第一級陸上無線技術士なので、技術系の質問をする。
データ・サイエンスに関しては
データ分析には必ず Python (pandas/scikit-learn) を使用してください。
数値計算手法の紹介の際には必ず例を載せてください。
特に指定がない限り、可視化は matplotlib か seaborn を使ってください。
ソースコードにはコメントを付与してください。
コードは、コピペしてそのままcursorで動く形式で出力してください。
質問の回答の数学及びソースコード背景についても、必要であれば補足してください。
数式の導出は省略せず書いて下さい。
私が知らなそうな数学や深層学習の専門単語はあたかも私が知っているように出さないで、一定の解説を専門用語の前に置いて欲しい。HOSVDとか
回答するときは回答前に自分でセルフレビューして間違っていないかどうかを確認してから解答して欲しい
回答する時は出典元情報の更新時間を確認して、情報が正確かどうか判断したのちに回答して
一回で膨大な量の回答は出さずに、こまめに出して欲しい。こちらから、次を要求する。詳細な回答を短めに出して欲しい。こちらから、その次を、要求する

🗑 クリア

保存

カスタム指示に書いた内容

専門分野・資格などの背景を伝える。

専門用語には説明を付ける。数式は途中の導出も示す。

回答前に見直し、出典や情報の更新時点を確認する。

一度に大量の回答を出さず、段階を分けて説明する。

使い分けの例：狭い技術上の疑問は検索で確認し、複数の研究・手法を比較する場合は詳しい調査を依頼する。

3.3 Cursor

AI エージェントを統合した開発環境として利用した。Python による開発、ChatGPT が作成したサンプルコードや実行形式問題集のデバッグ、コードレビュー、リファクタリングなどに利用した。

特に便利だったのは、コードの修正だけでなく、ターミナル操作を含む開発環境へのアクセス、複数ファイルの整理、共通処理の分離まで依頼できることだった。

例 1：人間がインストールコマンドを調べて実行する代わりに、「matplotlib を入れて」と指示することで、必要なパッケージのインストールを任せられることができる。

例 2：対象コードを指定して「バグを直して」と指示すると、エラー内容や周辺コードを確認しながら修正できる。

例 3：複数ファイルを指定して「共通部分を抜き出して Python ファイルとして作成して」と指示することで、重複していた処理を共通化できる。

そのため、自分でエラーメッセージを検索したり、パッケージのインストール方法を調べてコマンドを貼り付けたりする作業が減り、「コードが実行できない状態で長時間止まる」ことが少なくなった。

また、人間が手作業で行う必要性の低い整理作業を AI に任せ、その間に別の作業を進めることもできた。

利用したプランはCursor Proで、利用上限付きではあるが、今回の個人開発では十分なリソースを確保できた。利用状況の確認時点では、Cursor Modelsが43%使用、Other Modelsが33%使用であり、利用期間終盤でも一定の余裕があった。

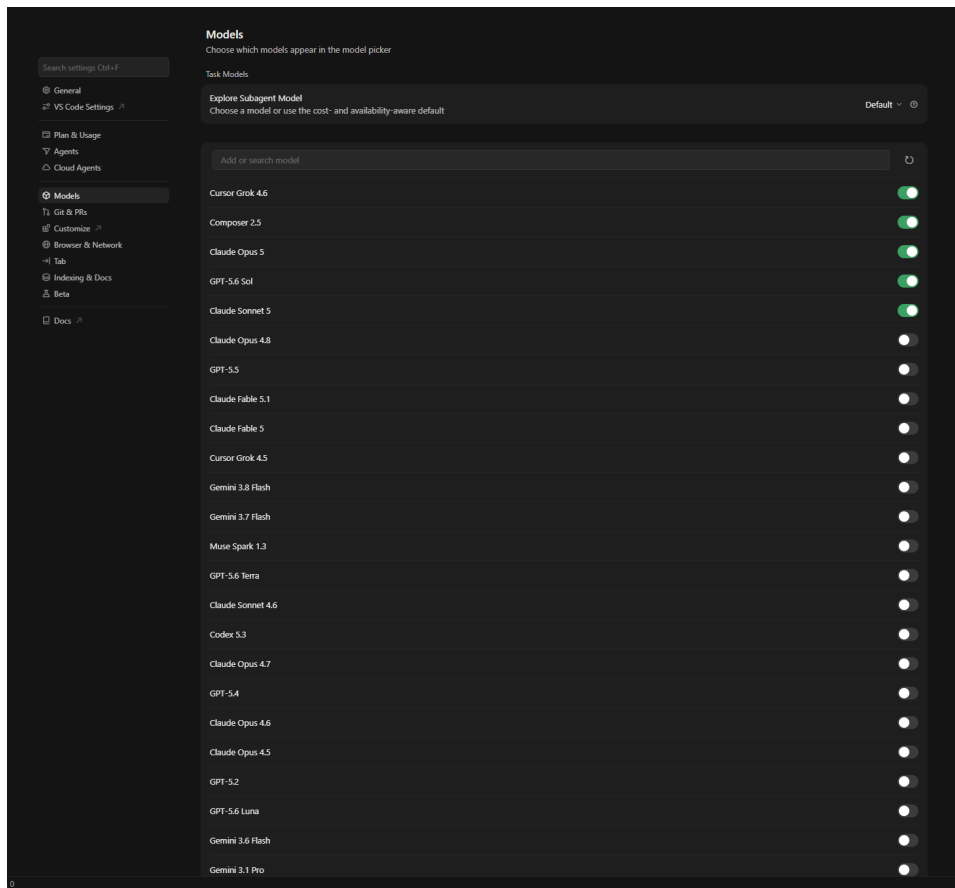
3.3 Cursor — 編集画面とAIエージェントを一緒に使う

コードを書く・直すだけでなく、複数ファイルの整理、実行、テストなどにも利用した。

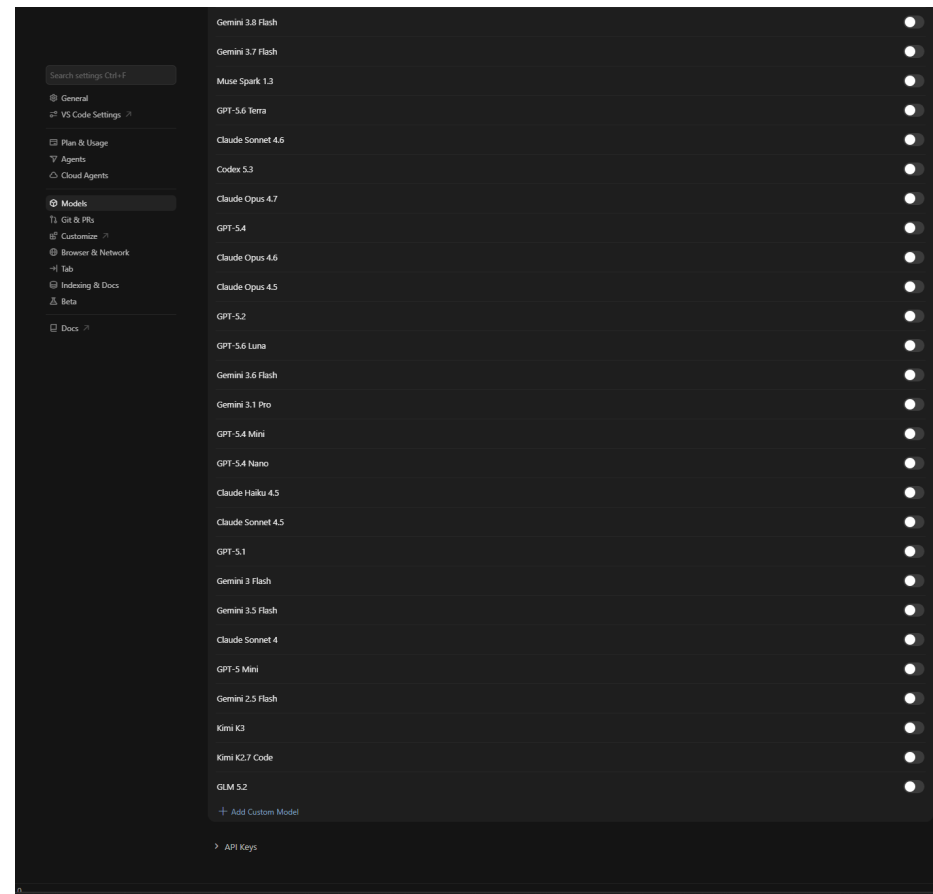


図4 Cursorのメイン画面。Autoは自動選択。メニューを開いて特定のモデルも選べる。

3.3 Cursor — 使用するモデルを設定する



モデル設定画面（上側）



モデル設定画面（下側）

主に有効にしているモデル : Cursor Grok 4.6 / Composer 2.5 / Claude Opus 5 / GPT-5.6 Sol / Claude Sonnet 5

設定で有効にしたモデルを、AIEージェント入力欄のモデル選択メニューに表示する。
一覧の灰色スイッチは「表示設定がオフ」を示し、プラン上の利用禁止とは限らない。

3.3 Cursor — 添付画面に並ぶモデルの例

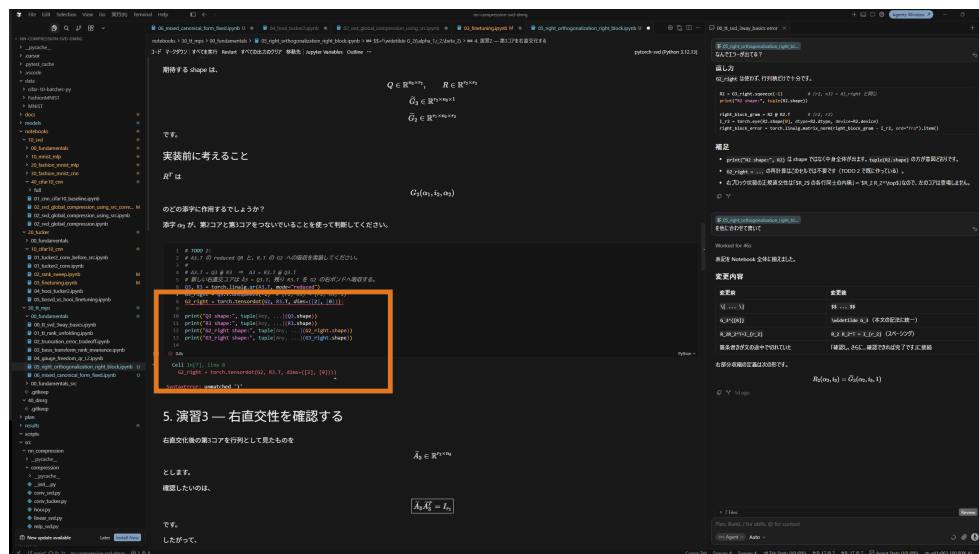
前ページのモデル設定画像1・2に並んでいる名称を、読みやすい文字で示す。

Cursor Grok 4.6	オン	GPT-5.6 Terra	オフ	Claude Haiku 4.5	オフ
Composer 2.5	オン	Claude Sonnet 4.6	オフ	Claude Sonnet 4.5	オフ
Claude Opus 5	オン	Codex 5.3	オフ	GPT-5.1	オフ
GPT-5.6 Sol	オン	Claude Opus 4.7	オフ	Gemini 3 Flash	オフ
Claude Sonnet 5	オン	GPT-5.4	オフ	Gemini 3.5 Flash	オフ
Claude Opus 4.8	オフ	Claude Opus 4.6	オフ	Claude Sonnet 4	オフ
GPT-5.5	オフ	Claude Opus 4.5	オフ	GPT-5 Mini	オフ
Claude Fable 5.1	オフ	GPT-5.2	オフ	Gemini 2.5 Flash	オフ
Claude Fable 5	オフ	GPT-5.6 Luna	オフ	Kimi K3	オフ
Cursor Grok 4.5	オフ	Gemini 3.6 Flash	オフ	Kimi K2.7 Code	オフ
Gemini 3.8 Flash	オフ	Gemini 3.1 Pro	オフ	GLM 5.2	オフ
Gemini 3.7 Flash	オフ	GPT-5.4 Mini	オフ		
Muse Spark 1.3	オフ	GPT-5.4 Nano	オフ		

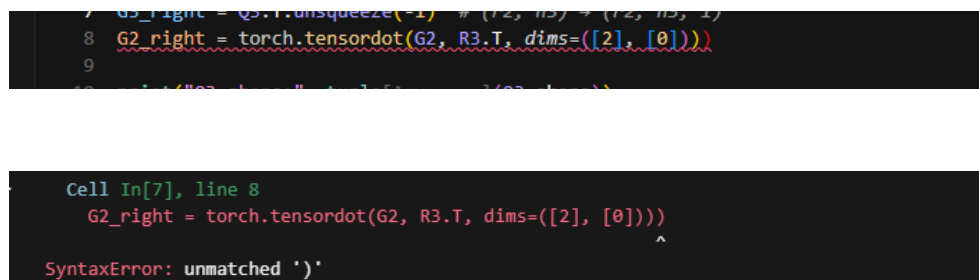
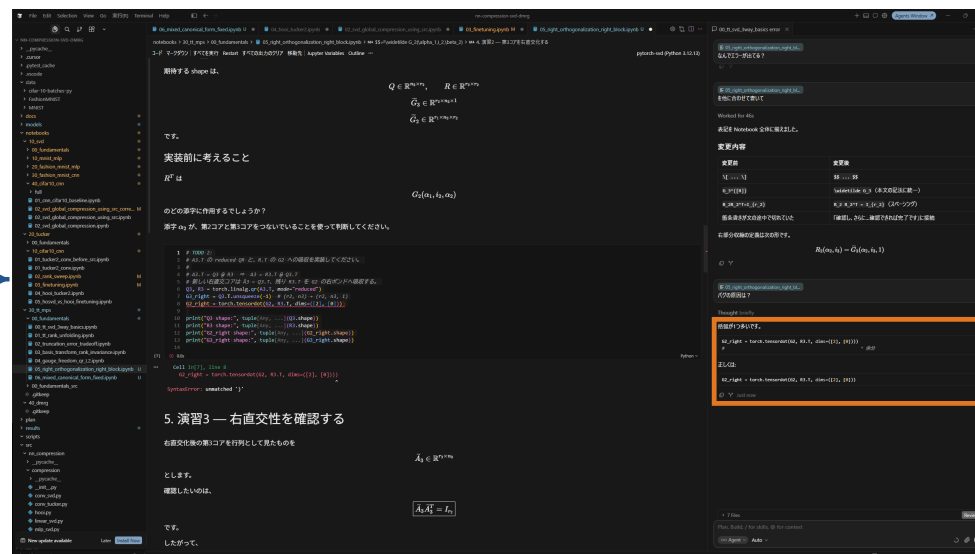
オン／オフは、添付画面でのモデル選択メニューへの表示設定。モデルや利用条件は変更される。

3.3 Cursor — バグ修正 ① エラー発生 → ② 原因を質問

① 赤い波線があり、実行時にエラー



② AIエージェントに原因を聞く



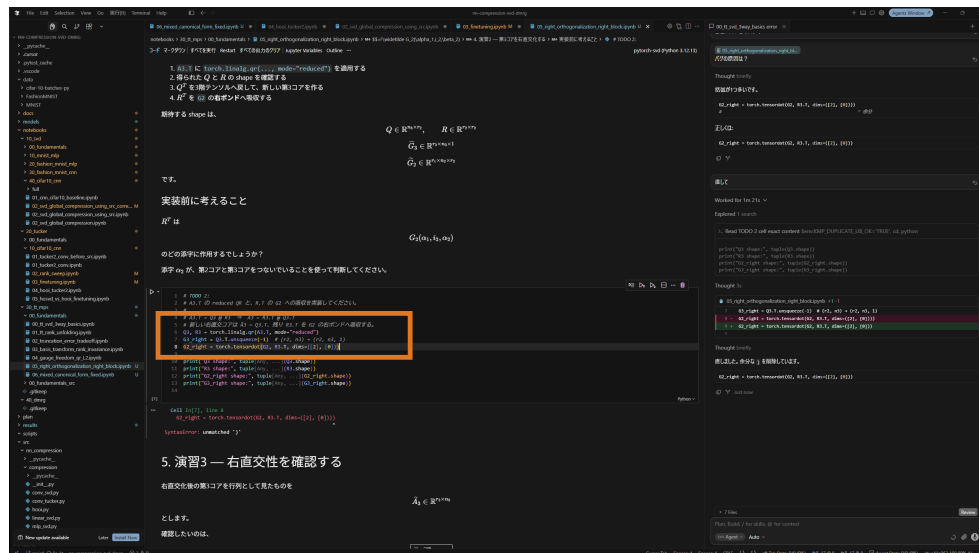
コードの赤い波線と、下のエラー出力を確認。



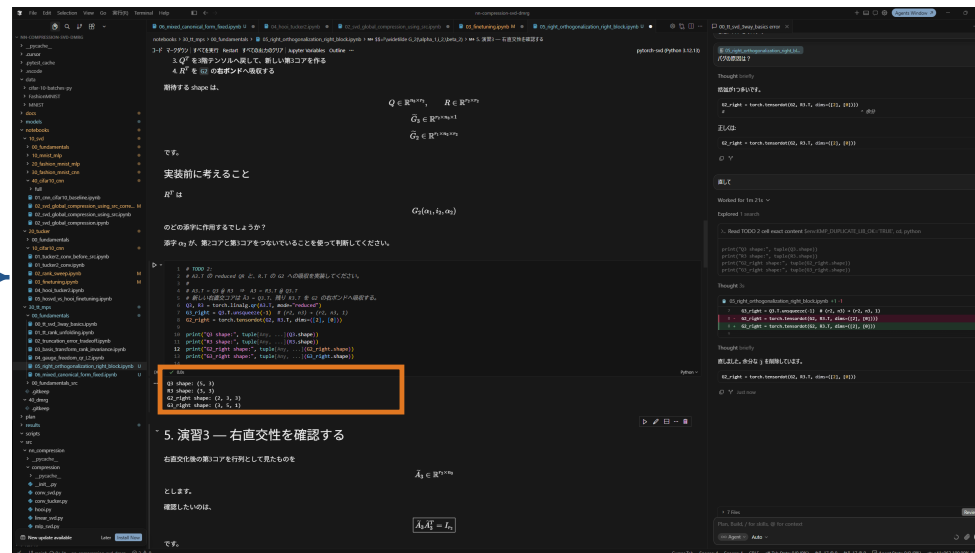
「バグの原因は？」と聞くと、余分な括弧があると説明が返ってきた。

3.3 Cursor ― バグ修正 ③ コード修正 → ④ 実行成功

③ AIに「直して」と依頼して修正



④ 修正したセルを再実行して成功



```

5 # 新しい右直交交点は  $\hat{A}_3 = Q_3.T$ 、残り  $R_3.T$  を  $G_2$  の右ボンドへ吸収する。
6  $Q_3, R_3 = \text{torch.linalg.qr}(A_3.T, \text{mode}=\text{"reduced"})$ 
7  $G_3\_right = Q_3.T.\text{unsqueeze}(-1) \# (r_2, n_3) \rightarrow (r_2, n_3, 1)$ 
8  $G_2\_right = \text{torch.tensordot}(G_2, R_3.T, \text{dims}=[(2), [0]])$ 
9
10 print("Q3 shape:", tuple[Any, ...](Q3.shape))

```

```

13 print("G3_right shape:", tuple[Any, ...](G3_right.shape))
14
[0] ✓ 0.0s
...
Q3 shape: (5, 3)
R3 shape: (3, 3)
G2_right shape: (2, 3, 3)
G3_right shape: (3, 5, 1)

```

余分な括弧を削除。下には修正前のエラー出力が残っている。

再実行後、エラーがなくなり、配列の形状が出力された。

3.3 Cursor — 学習結果のグラフ作成を依頼する

① グラフ用コードを書く前

The screenshot shows the Cursor IDE with a Python file named `fit_with_early_stopping.py`. The code includes a table of results and a section for saving the table and models as CSV.

epoch	train_loss	train_acc	val_loss	val_acc	early_stopping	best_val_acc
0	1.000000	0.000000	1.000000	0.000000	True	0.000000
1	0.999999	0.000000	0.999999	0.000000	True	0.000000
2	0.999998	0.000000	0.999998	0.000000	True	0.000000
3	0.999997	0.000000	0.999997	0.000000	True	0.000000
4	0.999996	0.000000	0.999996	0.000000	True	0.000000
5	0.999995	0.000000	0.999995	0.000000	True	0.000000
6	0.999994	0.000000	0.999994	0.000000	True	0.000000
7	0.999993	0.000000	0.999993	0.000000	True	0.000000
8	0.999992	0.000000	0.999992	0.000000	True	0.000000
9	0.999991	0.000000	0.999991	0.000000	True	0.000000
10	0.999990	0.000000	0.999990	0.000000	True	0.000000
11	0.999989	0.000000	0.999989	0.000000	True	0.000000
12	0.999988	0.000000	0.999988	0.000000	True	0.000000
13	0.999987	0.000000	0.999987	0.000000	True	0.000000
14	0.999986	0.000000	0.999986	0.000000	True	0.000000
15	0.999985	0.000000	0.999985	0.000000	True	0.000000
16	0.999984	0.000000	0.999984	0.000000	True	0.000000
17	0.999983	0.000000	0.999983	0.000000	True	0.000000
18	0.999982	0.000000	0.999982	0.000000	True	0.000000
19	0.999981	0.000000	0.999981	0.000000	True	0.000000
20	0.999980	0.000000	0.999980	0.000000	True	0.000000
21	0.999979	0.000000	0.999979	0.000000	True	0.000000
22	0.999978	0.000000	0.999978	0.000000	True	0.000000
23	0.999977	0.000000	0.999977	0.000000	True	0.000000
24	0.999976	0.000000	0.999976	0.000000	True	0.000000
25	0.999975	0.000000	0.999975	0.000000	True	0.000000
26	0.999974	0.000000	0.999974	0.000000	True	0.000000
27	0.999973	0.000000	0.999973	0.000000	True	0.000000
28	0.999972	0.000000	0.999972	0.000000	True	0.000000
29	0.999971	0.000000	0.999971	0.000000	True	0.000000
30	0.999970	0.000000	0.999970	0.000000	True	0.000000

The code includes a section for saving the table and models as CSV:

```
def save_results(results_dir, results_dir, fit_with_early_stopping):  
    # Save results to CSV  
    results_csv_path = results_dir / "results.csv"  
    df = pd.DataFrame(results)  
    df.to_csv(results_csv_path, index=False)  
    print(f"Results saved to {results_csv_path}")  
  
    # Save models to CSV  
    models_csv_path = results_dir / "models.csv"  
    df_models = pd.DataFrame(models)  
    df_models.to_csv(models_csv_path, index=False)  
    print(f"Models saved to {models_csv_path}")  
  
    # Save validation accuracy to CSV  
    val_acc_csv_path = results_dir / "val_acc.csv"  
    df_val_acc = pd.DataFrame(val_acc)  
    df_val_acc.to_csv(val_acc_csv_path, index=False)  
    print(f"Validation accuracy saved to {val_acc_csv_path}")
```

② AIがコードを追加し、実行するとグラフが表示

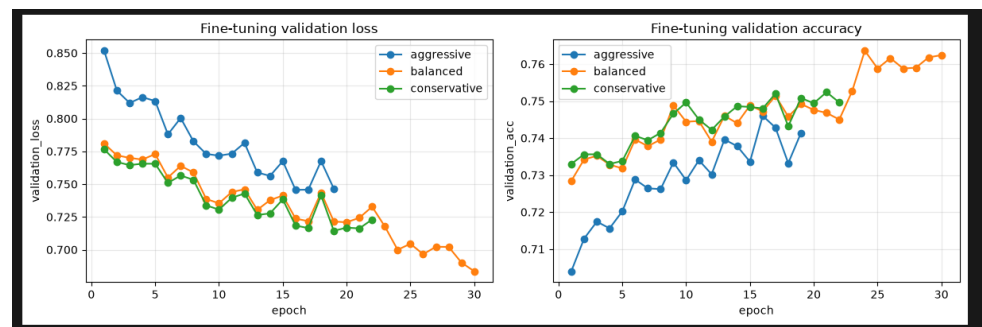
The screenshot shows the Cursor IDE with a Python file named `fit_with_early_stopping.py`. The code includes a section for plotting the training and validation loss and accuracy.

```
def plot_results(results_dir, results_dir, fit_with_early_stopping):  
    # Plot results  
    results_csv_path = results_dir / "results.csv"  
    df = pd.DataFrame(pd.read_csv(results_csv_path))  
    df.to_csv(results_csv_path, index=False)  
    print(f"Results saved to {results_csv_path}")  
  
    # Plot training and validation loss  
    plt.figure(figsize=(10, 5))  
    plt.plot(df["epoch"], df["train_loss"], label="train_loss")  
    plt.plot(df["epoch"], df["val_loss"], label="val_loss")  
    plt.legend()  
    plt.savefig(results_dir / "train_val_loss.png")  
    print(f"Training and validation loss plot saved to {results_dir / 'train_val_loss.png'}")  
  
    # Plot training and validation accuracy  
    plt.figure(figsize=(10, 5))  
    plt.plot(df["epoch"], df["train_acc"], label="train_acc")  
    plt.plot(df["epoch"], df["val_acc"], label="val_acc")  
    plt.legend()  
    plt.savefig(results_dir / "train_val_acc.png")  
    print(f"Training and validation accuracy plot saved to {results_dir / 'train_val_acc.png'}")
```

6. 学習曲線

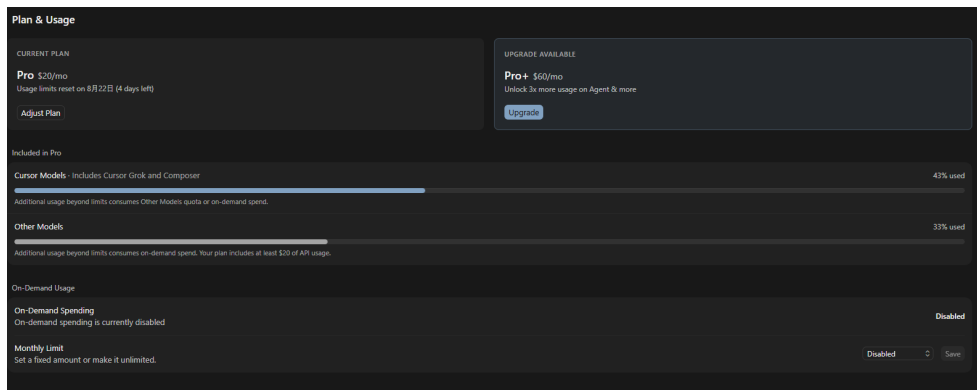
各候補の Early Stopping 用 validation 指標 (fit_with_early_stopping の履歴)。

1. 0.000000



「前のセルで計算した学習曲線をグラフ化して」と依頼。AIがグラフ用コードを記入し、セルを実行すると結果が見える。

3.3 Cursor — 利用状況とCodexレビューへの対応



2026年8月の確認時点：Cursor Models 43%使用、Other Models 33%使用。
追加の従量課金は無効。

今回の個人開発では、期間終盤も利用枠に余裕があった。

「パッケージを入れて」「共通処理をPythonファイルにまとめて」など、
開発環境の操作や整理も依頼した。

Codexのレビューで示された指摘内容をもとに、Cursorへ修正を依頼した。以下は実際の対応ログの要約。

Codexレビュー指摘に対するCursorの対応

- ① 指摘をコード・既存Notebookと照合し、修正範囲を整理。
- ② 共通コードや評価処理を修正。比較条件と乱数の扱いをそろえた。
- ③ テストを追加・実行し、49件すべて成功。
- ④ 原本を残したまま、修正版Notebookを別に作成。
- ⑤ その後、修正版を実行し、結果ファイル・グラフなどを確認。
- ⑥ 変更をGitへ記録。CIFAR-10実行後の対応ログでは2件のコミットを作成し、その時点ではpushしていない。

3.4 Cursor iOS（予定）

Cursor iOS はまだ利用していない。今後、スマートフォンから Agent へ指示して開発作業を継続する用途で試す予定である。

GitHub 上のコード修正依頼や Agent の進捗確認・追加指示を想定し、PC を使えない移動時間や昼休みの補助として活用できるか確認する。

PC では Cursor で本格的な実装・デバッグ・コード整理を行い、モバイルでは Cursor iOS から Agent へ指示する役割分担を想定している。

ただし、スマートフォンでは差分確認が難しいと考えられるため、必要に応じてタブレットなど画面の大きい端末も検討する。

GitHub を共通の保存先とし、端末が変わっても同じ成果物に対して作業を継続できるか確認する予定である。

3.5 Codex

Codex は、GitHub 上のコードを対象に、コード確認、修正、コードレビューなどを行う用途で利用した。

PCからもモバイルからも指示できる。今回の利用形態ではモバイル利用時にもPC側の環境が必要だった。Cursor iOSは未利用のため、モバイルでの操作性比較はまだ行っていない。

コードレビューでは、実装を行った Cursor にそのまま自己確認させるだけでなく、別の AI サービスである Codex へレビューを依頼する使い方を試した。Codex のレビュー結果や改善案をもとに Cursor で修正・テストを行うことで、レビュー担当と修正担当を分けた。

この流れにより、「GitHub上のコード → Codexへレビュー依頼 → Cursorで修正 → GitHubへ反映 → Codexで再確認」という役割分担を実際に行うことができた。

Codex は ChatGPT Plus 契約内で利用した。利用状況の確認時点では週次利用枠の残りが 79%と表示されていた。重いコードレビューを繰り返す場合は Cursor よりも利用枠を意識する必要があると感じた。

3.4 Cursor iOS（予定） / 3.5 Codex

Cursor iOSは未利用。スマホから作業依頼・進捗確認を試す予定で、実際の操作性はまだ比較していない。

Codex : PC・モバイルからコード確認やレビューを依頼。今回はCursorとは別の確認担当として使った。

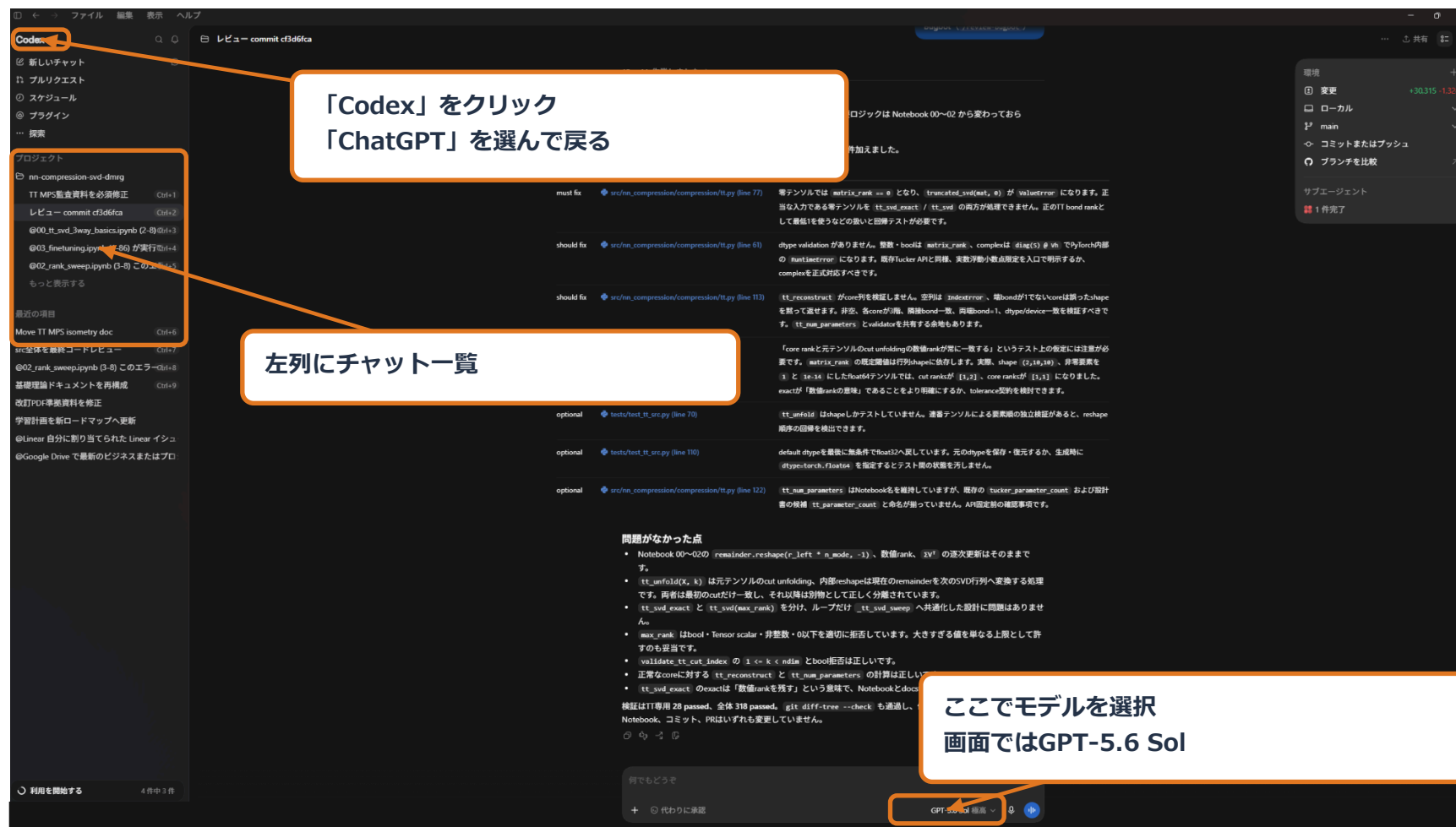


図5 Codex画面は9月の別レビュー事例。次ページの8月のC判定→B判定の記録とは区別する。

3.5 Codex — 利用枠とレビュー結果



2026年8月の確認時点：週次利用枠は79%残。
個々のレビューによる消費量は、この画像だけでは分からない。

GitHub上の成果物を確認し、問題の指摘・改善案・修正後の再確認に利用した。重いレビューでは利用枠を意識した。

レビューだけを行う条件を指定し、指摘をもとにCursorで修正・実行・テストを進めた。

Codexによるレビュー結果

1回目（2026年8月18日）：C判定。
比較条件やコードとNotebookの不整合など、結果の信頼性に関わる問題が見つかり、修正・再確認が必要。

Cursorでコード・比較条件・テスト・修正版Notebookを整理し、実行結果を用意。

2回目（2026年8月19日）：B判定。
修正版5本の実行結果を有効と判断。Criticalは0件、49件のテスト成功を再確認。再実験は不要。ただし、対応バージョンや成果物管理、文書など、確定前の整理は残った。

3.6 補助サービスの特徴

NotebookLM

与えた情報を整理・要約することが得意な AI サービスである。

Perplexity で調査した内容の取りまとめ、複数資料の統合・要約、Markdown 形式のノート作成などに利用した。大きいファイルや複数ファイルでも、まとめ方の方向性をうまく指定できれば、一つの文書として整理できる点が便利だった。

一方で、こちらが意図したまとめ方を伝えるまでに何度か指示が必要になる場合があった。利用したプランは無料プランであり、現状の用途では無料版で十分だと考えている。

3.6 補助サービス — NotebookLMで資料をまとめる

Perplexityで調べた内容や、複数の資料を渡し、要約・統合してノートの下書きを作る用途に利用した。

The screenshot displays the NotebookLM interface with three main panels: 'Sources' on the left, a central chat area, and 'Studio' on the right. Three numbered callouts provide a step-by-step guide:

- ① 左の「ソースを追加」
資料・Webページを読み込む**
This callout points to the '+ ソースを追加' button in the Sources panel. Below it, text explains that saved sources are displayed here and can be used for questions or added via file upload.
- ② 中央の入力欄で質問
追加した資料の要約・説明を依頼**
This callout points to the central chat input area, which prompts the user to ask questions or create content based on the added sources.
- ③ 右のStudioで
音声解説・レポートなどを作成**
This callout points to the Studio panel, which offers various output options like audio explanations, slide materials, reports, and more.

At the bottom of the interface, a note states: 'Gemini Notebook は不正な場合があります。回答は再確認してください。' (Gemini Notebook may be incorrect. Please reconfirm the answer.)

図6 添付画面は資料追加前（0個のソース）。資料を追加してから、その内容について質問する。

Google Colaboratory

GitHub と連携して Jupyter Notebook を実行できるクラウド実行環境として利用した。

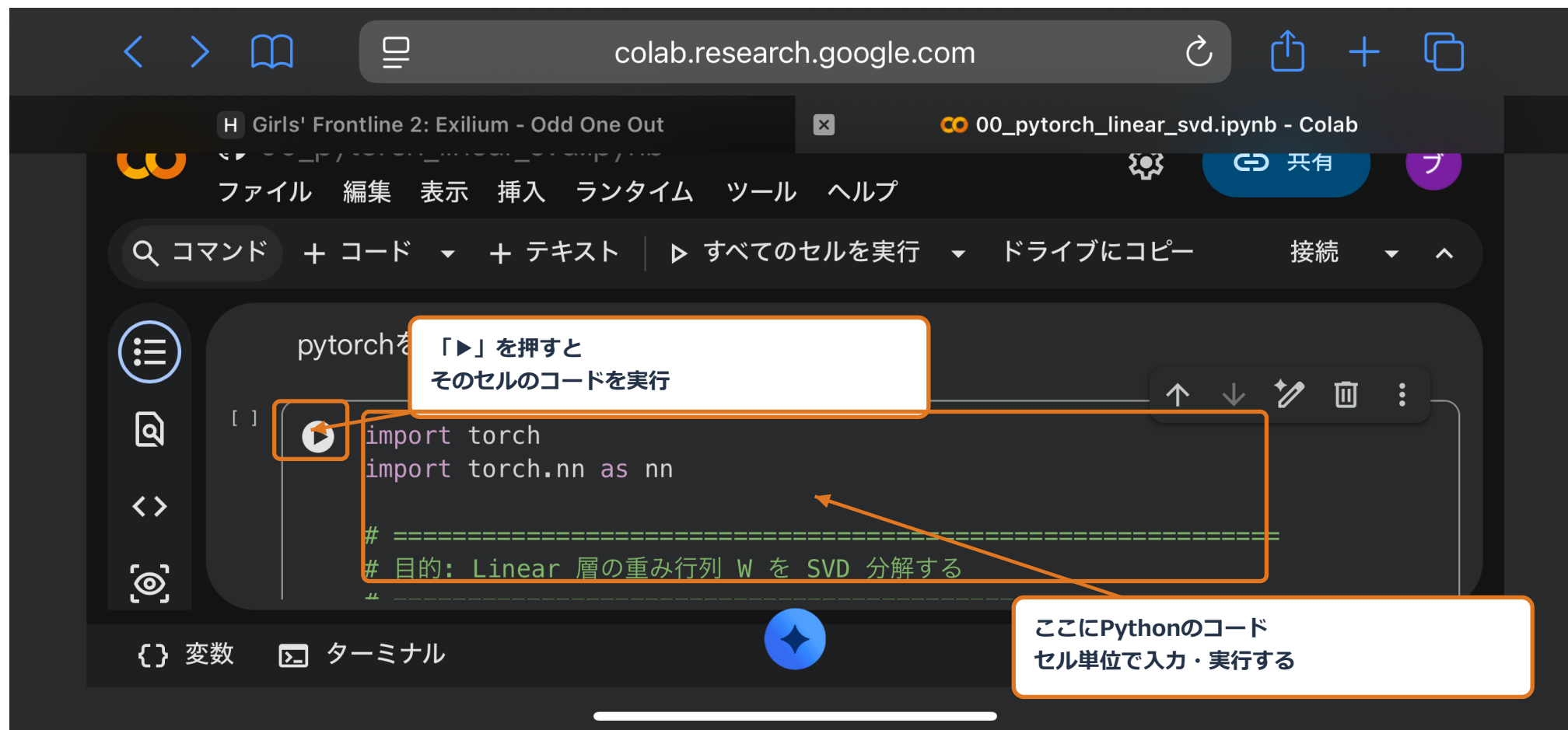
会社の昼休みにコーディング学習する用途で利用した。PC がなくてもブラウザから Notebook を実行できる点は便利だったが、スマートフォンでは画面が狭く、コード入力や実行結果の確認が難しい。

モバイル環境では、現在はCodexでAgentへ作業を依頼し、Google ColaboratoryでNotebookを実行・確認している。今後はCursor iOS（予定）も試し、用途を使い分ける予定である。

利用したプランは無料プランであり、現状の用途では無料版で十分である。

3.6 補助サービス — スマホでGoogle Colaboratoryを使う

ブラウザでNotebookを開き、Pythonを実行・確認する環境。会社の昼休みなどにスマホで使った。



実際に使って分かったこと

スマホでは画面が狭く、コード入力・修正や実行結果の確認が難しい。今回の使い方では、本格的な学習・開発には実用的ではなかった。

GitHub / Obsidian

GitHub

コード、Jupyter Notebook、Markdown ファイル、実験成果物などの構成管理・成果物管理に利用した。

今回の AI サービス連携では、単なる最終的な保存場所ではなく、PC・モバイル・複数の AI サービスをつなぐ開発成果物のハブとして利用した。

PCではCursorやCodex、モバイルではCodexとGoogle Colaboratoryから同じGitHub上の成果物を扱っている。今後はCursor iOS（予定）からも扱えるか確認する。

また、GitHub 上のコードを Codex や ChatGPT へレビュー依頼し、その結果を Cursor で修正するという AI サービス間の連携にも利用した。

利用したプランは無料プランである。今回の用途では Cursor / Codex を利用しているため GitHub 無料版で十分であり、別の開発環境を採用する場合には GitHub Copilot などを選択肢になると考えている。

Obsidian

Markdown ファイルを見やすく表示・編集・管理できるノート管理ツールとして利用した。

ChatGPT、NotebookLM などで作成した Markdown ノートを保存し、自分で内容を追記・修正する用途に利用している。AI が作成した文章を自分で読み直し、追記しながら知識を整理する最終的なノート環境として使用した。

また、GitHub に蓄積した学習内容や実験結果をもとに ChatGPT へ次の学習・開発方針を相談することで、知識管理から次の開発へつなげている。

利用したプランは無料プランであり、現状の用途では無料版で十分である。

4. AIサービス連携の実例

4.1 開発・レビューにおける連携例

前章では各サービス単体の特徴を説明した。本章では内容の重複を避けるため、実際の開発案件で複数 AI をどのように連携させたかを一つの事例として示す。

今回の技術学習では、ニューラルネットワークを軽量化するプログラムを題材として、複数の AI サービスを次のように分担して利用した。

1. ChatGPT で学習・開発計画を作成

約 2 年間の学習・研究ロードマップを作成し、何をどの順番で試すかを整理した。

2. Cursor で実装・整理

コードの実装、デバッグ、共通処理の整理、テストなどを行った。

3. GitHub へ成果物を集約

コード、Notebook、結果ファイルを GitHub で管理し、別の AI サービスからも同じ成果物を確認できるようにした。

4. ChatGPT で Codex 向けのレビュー依頼文を作成

レビュー対象、確認項目、変更禁止、問題の分類方法、最終判定方法まで依頼文に明記した。

4.1 開発・レビューにおける連携例（続き）

5. Codex でコードレビュー

1 回目のレビューでは C 判定となり、結果の信頼性に関わる複数の問題が指摘された。

6. Cursor でレビュー指摘を反映

Codexのレビューで指摘された内容を踏まえ、Cursorでコード修正、比較条件の統一、テスト、修正版 Notebook 作成、成果物整理を実施した。49 件のテストはすべて成功した。

7. Codex で再レビュー

修正後に再度レビューを行ったところ、B 判定となり、結果そのものは有効で再実行は不要と判断された。残った指摘は、公開・確定前の軽微な整理が中心だった。

8. ChatGPT で次の方針を整理

レビュー結果と実験結果をもとに、次に行う学習・開発内容を再度整理した。

この事例では、一つの AI にすべてを任せるのではなく、「計画・依頼文作成」「実装」「第三者レビュー」「修正」という役割を複数 AI に分けることができた。

4.2 連携によって得られた効果

実装担当とレビュー担当を分けることで、同じ AI による自己確認だけでは気付きにくい問題を発見できた。

Codexの指摘を踏まえてCursorに対応を依頼することで、レビュー結果を人間が一つずつコードへ反映する作業を減らすことができた。

レビューで問題が見つかった後も、Cursor が修正、テスト、実行確認まで連続して作業できた。

GitHub を共通のハブにすることで、PC・モバイル・複数 AI サービスから同じ成果物を扱うことができた。

AI が実装や調査を行っている間、人間側は別の作業を進めることができ、待ち時間を別タスクへ振り分けられた。

ChatGPT へ結果を戻すことで、個別作業で終わらず「次に何をするか」まで継続して整理できた。

4.3 利用上の課題

利用して分かった注意点

スマートフォンだけではコードの差分や長い Notebook を確認しづらく、モバイルで本格的に利用する場合はタブレットなど大きい画面の端末が欲しくなる。

複数の AI サービスを無計画に併用すると、サービス間の情報転記や指示の調整を人間が担う必要があり、人間が単なるサービス間の中継役になってしまう。

各サービスには利用上限があり、特に大規模なコードレビューや Agent 作業では利用量を意識する必要がある。

AI が出した回答やコードが正しいとは限らないため、最終的には利用者側が内容を理解し、採用してよいか判断する必要がある。

重要：AI サービスは数を増やすこと自体が目的ではなく、「どのサービスに何を任せるか」を先に決めて利用することが重要である。

5. 継続利用評価

サービス	推奨度	理由
ChatGPT	継続推奨	学習計画、技術相談、文書作成、依頼文作成など用途が広く、全体のハブとして使えるため
Cursor	継続推奨	コード生成だけでなく、デバッグ、実行、パッケージ管理、複数ファイル修正まで実開発をまとめて支援できるため
Cursor iOS（予定）	利用予定	PCを使えない場面でAgentへ指示し、作業を継続できるか検証する予定
Codex	継続利用	Cursorとは別のレビュー担当として利用でき、コード全体の第三者確認に有効だったため
Perplexity	条件付き推奨	専門技術・論文調査では有効だが、一般的な質問では他サービスと役割が重なるため
NotebookLM	無料版で十分	資料整理・要約には便利だが、現在の利用頻度では無料版で足りるため
Google Colaboratory	無料版で十分	モバイルでのNotebook実行に便利だが、現在の利用範囲では有料機能が不要なため
GitHub	無料版で十分	個人のコード・Notebook・成果物管理では無料版で足りるため
Obsidian	無料版で十分	Markdownノート管理・編集用途では無料版で足りるため

6. 今後の活用予定

今後も各サービスの役割を固定し、用途が重ならないように使い分ける。

サービス	今後の主な活用予定
ChatGPT	学習スケジュール管理 / 技術学習・開発方針の整理 / 学習ノート・成果物作成 / 他AIへの依頼内容整理
Perplexity	論文・専門技術調査 / 複数研究を横断したDeep Research / 数式や考え方の画像入力による確認
Cursor	Pythonコードの実装・デバッグ・整理 / 開発環境操作 / 他AIのレビュー指摘反映
Cursor iOS（予定）	モバイルからAgentへ修正指示・進捗確認を試す予定 / PCを使えない時間の補助として検証
Codex	GitHub上のコードレビュー / Cursorとは別視点での確認 / 修正後コードの再レビュー
NotebookLM / Obsidian	調査資料・学習内容の整理 / 長期的な知識蓄積

今後も「ChatGPT＝全体整理」「Perplexity＝調査」「Cursor＝実装」「Codex＝レビュー」を基本とし、必要に応じて補助サービスを組み合わせる。

7. 結論

今回の利用では、各サービスを次のように役割分担した。

Perplexity：専門技術・論文調査、数式・考え方の確認

ChatGPT：学習支援、技術相談、開発方針、教材・ノート・成果物作成、全体統括、他 AI への依頼文作成

Cursor：PC での本格的な実装、デバッグ、コード整理、開発環境操作

Cursor iOS（予定）：モバイルからAgentへの開発指示を試す予定

Codex：コードレビュー、修正案の提示、再確認

Google Colaboratory：モバイル環境での Notebook 実行・検証

GitHub：各開発環境・AI サービスをつなぐ開発成果物のハブ

NotebookLM / ChatGPT の長文文書作成機能：文書整理・長文ノート作成

Obsidian：知識管理

AI サービスは単体で利用するよりも、それぞれの得意分野に応じて役割を分担し、連携して利用することで効果が大きくなると感じた。

特に今回のコードレビューでは、ChatGPT がレビュー依頼を設計し、Codex が問題を指摘し、Cursor が修正を行い、Codex が再確認するという流れを実際に構築できた。これは、複数の AI サービスを単なるチャットツールとしてではなく、開発工程ごとの担当者のように利用した例である。

また、AI サービスが処理を行っている間、人間側は別のタスクを進めることができるため、作業時間の使い方にも変化があった。

一方で、複数サービスが無計画に使うと、人間が AI サービス間の情報を運ぶだけの存在になり、かえって作業が増える可能性がある。そのため、GitHub のような共通の成果物置き場を利用し、各サービスの役割を明確にすることが重要である。

さらに、AI の出力をそのまま信頼するのではなく、最終的には利用者自身が内容を理解し、正誤や採用可否を判断できることが必要である。

すべての AI サービスを有料契約する必要はなく、日常的に使う ChatGPT・Cursor を中核とし、専門調査が多い場合に Perplexity Pro を利用するなど、用途に応じて有料版と無料版を使い分ける方法が適していると考える。